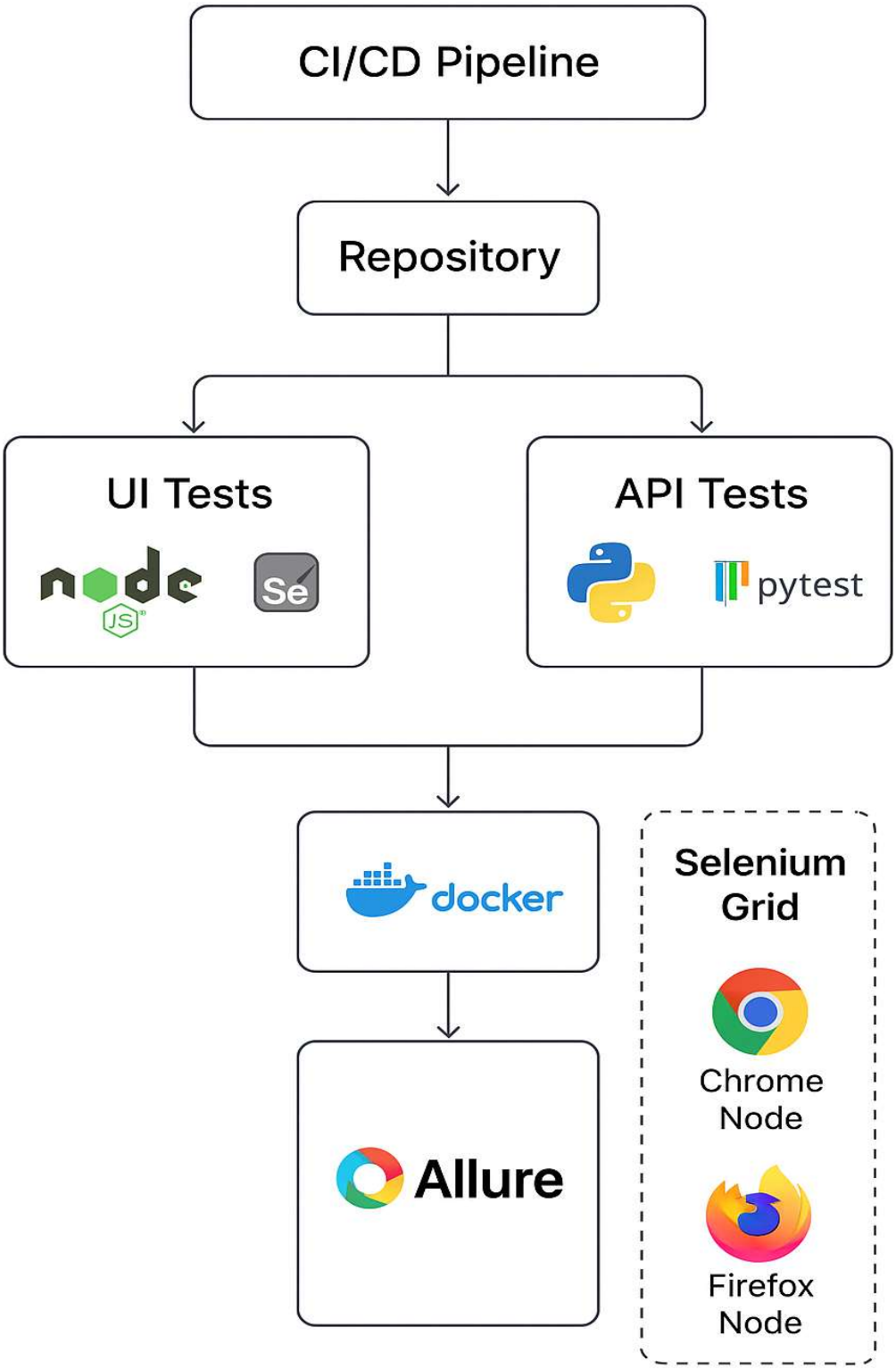
CI/CD Strategy and enterprise level installation



1. Executive Summary

The QA Automation Engineering Team requests a robust infrastructure that supports continuous integration, continuous delivery, and scalable automated test execution. This environment will enable:

- Parallel UI and API test execution
- CI-driven build and test automation
- Containerized execution (Docker/Kubernetes)
- Selenium Grid for multi-browser UI testing
- Enterprise-grade security, monitoring, and storage
- Future scalability for additional teams and test suites

This request outlines the compute, storage, network, and security requirements needed to implement and operate a production-ready automation ecosystem.

2. Business Justification

Modern development workflows require automated testing integrated with CI/CD pipelines. Current systems are inadequate for the scale, speed, and reliability required by multiple development teams.

Benefits include:

- 40–70% reduction in regression execution time
- Rapid feedback loops for developers and testers
- Improved product stability and reduced escape defects
- Standardized platform for automation across UI and API layers
- Scalable execution that grows with team size and complexity

Failure to provision infrastructure will result in:

- Slow-release cycles
- Manual regression bottlenecks
- Increased production incidents
- Inconsistent environments for testing

3. Overview of Required Architecture

Infrastructure requested includes:

- CI/CD Orchestration (Jenkins, GitLab CI, or Azure DevOps)
- Worker node pool for parallel test execution
- Kubernetes cluster for Selenium Grid + test execution jobs
- Browser nodes (Chrome/Firefox)
- Shared storage for logs, artifacts, and Allure reports
- Secure secrets management
- Networking and access to internal environments
- Logging, monitoring, and alerting integrations

4. Detailed Infrastructure Requirements

4.1 CI/CD Orchestration Layer

Purpose: Trigger builds, execute pipelines, store historical results, and manage distributed execution.

Requested Resources

- **2 CI Controller nodes (high availability)**
    - CPU: 8 vCPUs each
    - RAM: 32 GB each
    - Storage: 500 GB SSD each
    - OS: Linux (RHEL)
- Load balancer for HA controllers.

Responsibilities of CI Controllers

- Manage pipelines (UI + API automation)
- Publish Allure reports.
- Distribute build jobs to runners.
- Retain logs and artifacts.

4.2 CI/CD Runner / Compute Pool

Purpose: Execute automation tasks in parallel.

Requested Initial Runner Pool

- **6 Runner VMs**

    - CPU: 16 vCPUs
    - RAM: 32 GB
    - Storage: 300 GB SSD
    - Docker pre-installed

Autoscaling Requirements

- Autoscale runner pool up to **20 additional nodes** during heavy load.
- Nodes join/leave automatically based on job queue length.

4.3 Kubernetes Cluster (Test Execution + Selenium Grid)

Cluster Overview

- Hosts Selenium Grid (hub + event bus)
- Runs dynamic Selenium browser nodes (Chrome/Firefox)
- Executes WebdriverIO and Pytest job containers.

Kubernetes Cluster Requirements

Control Plane

- Managed service OR 3x 4 vCPU / 16 GB RAM nodes

Worker Nodes (Browser & Test Execution Nodes)

- 6 initial workers:
    - CPU: 16 vCPUs
    - RAM: 64 GB
    - Storage: 1 TB SSD
- Autoscaling enabled to add more browser nodes

Browser Node Requirements

- Chrome Node image and Firefox Node image
- Each browser node:
    - CPU: 4 vCPUs
    - RAM: 8–16 GB
    - Storage: 40 GB
- Initial count: 10 nodes

- Scalable up to 50 nodes

5. Storage Requirements

Artifact & Log Storage

- Minimum: **5 TB** S3-compatible or NAS storage
- Used for:
    - Test logs
    - Screenshots & videos
    - Allure report histories
    - Test artifacts (JUnit XML, JSON logs)

Retention:

- 90 days for active logs
- 1 year for archived runs

Backup Requirements

- Daily incremental backups
- Weekly full backups
- Monthly offsite archival (optional)

6. Network & Security Requirements

6.1 Network Access

The CI/CD and test execution infrastructure must have secure access to:

- Internal development/test environments (DEV, QA, UAT)
- Application under test (web UI and APIs)
- Internal artifact repositories
- Internal container registry

Firewall rules required:

- Allow CI/Runners → Internal UAT/QA environments (HTTPS)
- Allow Runners → Kubernetes cluster API.
- Allow Selenium Nodes ↔ Selenium Hub

6.2 Secrets Management

- All credentials stored in enterprise vault:
    - HashiCorp Vault
    - Azure Key Vault
    - AWS Secrets Manager
- No plaintext credentials in code or pipelines

6.3 Identity & Access Control

- Create service accounts for:
    - CI controllers
    - CI runners
    - Kubernetes deployments
- Role-based access (RBAC)
- Least privilege access.

7. Logging, Monitoring, and Alerting

Required Tools

- **Monitoring:** Prometheus + Grafana
- **Logging:** ELK / EFK stack or enterprise logging tool
- **Alerting:** Email, Teams, or Slack

Use Cases

- Track CPU, RAM usage of runners and browser nodes.
- Debug Selenium node failures.
- Monitor CI queue length and health.

8. Software Stack (IT-Installed or Approved)

Required Software Platforms

- Docker Engine
- Kubernetes (managed or self-hosted)
- Node.js (latest LTS)
- Python 3.13+ (latest GA version)

- Selenium Grid 4.x
- WebdriverIO CLI
- Pytest
- Allure (latest GA version)
- Internal Git platform (GitHub Enterprise / GitLab / Bitbucket)
- Internal Container Registry

9. Scalability Plan

Phase 1 – Initial Deployment

- 10 UI parallel sessions
- API CI runs on-demand.
- Limited browser diversity

Phase 2 – Mid Scale

- 50 parallel UI sessions
- Support for multiple teams
- Increased storage to 15 TB

Phase 3 – Enterprise Scale

- 200+ parallel sessions
- Cross-browser matrix expansion
- Multiple CI/CD pipelines integrated with microservices.

10. Risk Assessment

Risks of Under-Provisioning

- Long test cycles → slower releases
- Pipeline instability → developer frustration.
- Increased production failures

Risks of Over-Provisioning

- Higher initial cost (mitigated by autoscaling)

Risk Mitigations

- Autoscaling keeps costs proportional to usage.

- Monitoring ensures early detection of capacity issues.
- Infrastructure supports horizontal expansion.

11. Acceptance Criteria

The infrastructure is approved and accepted when:

1. UI tests run in parallel with <1% flakiness.
2. API tests (~200) run in <5 minutes for smoke suite.
3. CI pipelines run and complete validations in <20 minutes.
4. Allure reports generated and available internally.
5. Selenium Grid stable under load for ≥ 2 hours.
6. All passwords stored in enterprise vault.

12. Requested Timeline

- **Week 1–2:** Provision CI controllers + runners
- **Week 3–4:** Deploy Kubernetes cluster.
- **Week 5:** Set up Selenium Grid and browser nodes.
- **Week 6:** Integrate pipelines, test execution.
- **Week 7–8:** Monitoring, tuning, documentation

Total: **8 weeks for full production readiness**

If additional detail, technical architecture diagrams, or cost breakdowns are needed, the QA Automation Engineering Team is available to supply further documentation.